`product` should say it returns an exact catalogue record by SKU. Include constraints in descriptions where they help selection, but enforce those constraints in code as well.

Next create a ten-question routing test. Include queries requiring no tool, one tool and two tools. Score whether the model selects the right operation before judging the final prose. A beautifully phrased answer reached through the wrong tool is still a system failure.

Build a simple agent loop

At this point we can use the word agent without hand-waving. Our agent is a loop in which the model can inspect the current conversation, request an allowed tool, observe the result and choose its next step.

The core logic can remain small:

```
for step in range(MAX_STEPS):
        response = ask_model(messages, tools=tool_schemas)

      if not response.tool_calls:
          return response.text

      for call in response.tool_calls:
        result = execute_validated_tool(call)
         messages.append(call.as_message())
             messages.append(tool_result_message(result))

  raise RuntimeError("Agent exceeded step limit")
```

Real implementations require model-specific parsing, but the design is more important than a framework.

Set `MAX_STEPS` to something conservative such as five for the first build. Without a limit, a confused model may repeatedly search for the same information or bounce between tools. Track tool names and arguments, and stop obvious duplicate calls unless repetition is legitimate.

Add timeouts. A web API, database or local script can hang. Catch exceptions and return a controlled tool error that the model can explain or recover from. Do not expose stack traces, passwords or connection strings in the tool result.

Add a budget. You can cap database queries, expensive API calls or the total rows processed during one request. Local AI is not automatically cost-free: CPU time, GPU time, network services and human attention are resources too.

Most importantly, distinguish read operations from write operations. Searching files or reading stock is relatively low consequence. Deleting a file, sending a message, placing an order or modifying a customer record changes the outside world. The latter should generally require a user confirmation immediately before execution.

A practical flow is:

Plan → retrieve/read → show proposed action → user confirms → execute write

Confirmation should be concrete. "Send this draft to accounts@example.com?" or "Rename these 12 files?" tells the user what will happen.

Do not confuse more autonomy with a better product. For many applications, a one- or two-tool workflow with explicit approval is faster to debug and safer to trust than a twenty-step autonomous agent.

Connect tools through MCP

Our Python dictionary works well inside one application, but every AI program inventing its own connector format quickly becomes repetitive. Model Context Protocol, or MCP, addresses that integration problem.

MCP is an open standard for connecting AI applications to the systems where tools and data live. The current TypeScript SDK describes the basic arrangement plainly: an MCP server can expose tools, resources and prompts, while an MCP host connects to that server and makes those capabilities available to an AI application.

The July 28, 2026 specification also introduced a stateless protocol core, stronger authorisation mechanisms, cacheable discovery results and extensions for additional workflows. The maintainers report that its Tier 1 SDKs are now seeing close to half a billion downloads a month, illustrating how quickly the protocol has become part of the agent ecosystem.

The terminology is worth separating:

Tools perform operations: look up an order, run a calculation or update a record.

Resources expose readable context: a document, database-backed record or another piece of information identified by the server.

Prompts provide reusable prompt templates or workflows supplied by the server.

Instead of teaching your assistant the custom details of every integration, an MCP client can discover what an authorised server offers and invoke capabilities through a common protocol.

Do not replace working local functions merely because MCP exists. Understand direct tool calling first; MCP then becomes an interface choice rather than magic.

To experiment, install an official MCP SDK in a separate test project and create a tiny local server exposing a read-only tool such as `lookup_product`. Give the tool an explicit input schema and return structured data. Connect from an MCP-capable client and verify that the tool is discovered and called.

The 2026 MCP revision formalises JSON Schema use for tool inputs and outputs, and current SDK documentation emphasises validating structured results against declared output schemas.